

Lab 2: Expressions and Operators

Computer Programming () |

Duration: 2 hours (in-lab) | **Topic:** Arithmetic, formatted output, integer vs. floating-point

Objectives

- Build expressions with arithmetic, relational, logical, and assignment operators.
- Understand operator precedence and the difference between integer and floating-point division.
- Use `printf` field width, alignment, and base specifiers.
- Use the modulus operator to extract digits.

Quick Reference

Arithmetic. `+` `-` `*` `/` `%` Integer `/` truncates toward zero: `7/2 == 3`, `-7/2 == -3`.

Modulus. `a % b` is the remainder; defined only for integers.

Mixed types. If either operand is floating-point, the result is floating-point: `7 / 2.0 == 3.5`.

Precedence (high $\rightarrow$ low). `()` `>` unary `-` `>` `*` `/` `%` `>` `+` `-` `>` `<` `<=` `>` `>=` `>` `==` `!=` `>` `&&` `>` `||` `>` `=`.

printf format. `%5d` = width 5, right-aligned; `%-5d` = left-aligned; `%05d` = zero-padded; `%.2f` = 2 decimals; `%o/%x` = octal/hex.

Character math. `'A'` = 65, `'a'` = 97. So `'a' - 'A'` == 32.

In-Lab Exercises (target: 2 hours)

In-Lab Exercise 1: Formatted Output (25 min)

Print the six integers 123, 451, 332, 271, 74, 54 as a 2×3 grid. Each cell must use a field width of 5 and be *left-aligned*. **Hint:** `printf("%-5d%-5d%-5d\n", ...)`.

In-Lab Exercise 2: Base Conversion (20 min)

Read a non-negative integer from the user and print its value in decimal, octal, and hexadecimal (uppercase) on three separate lines. Use `%d`, `%o`, `%X`.

In-Lab Exercise 3: Sum of Digits (3-digit number) (35 min)

Read a 3-digit positive integer (between 100 and 999) and print the sum of its three digits. **Approach:** extract each digit with `n % 10` and `n / 10`. **Example:** input 479 $\Rightarrow$ output 20.

In-Lab Exercise 4: Average of Four Real Numbers (20 min)

Read four `double` values and print their average with three decimal places. **Pitfall:** use floating-point types and `%lf` in `scanf`.

In-Lab Exercise 5: Swap Without a Helper Variable (20 min)

Read two integers `a` and `b`, then swap them *without* using a third variable (use arithmetic: `a = a + b`; `b = a - b`; `a = a - b`;). Print the values before and after the swap.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Integer vs. floating-point division

Predict the printed output:

```
#include <stdio.h>
int main(void) {
    int    a = 9, b = 4;
    double x = 9, y = 4;
    printf("%d %d\n", a / b, a % b);
    printf("%.2f\n", x / y);
    printf("%.2f\n", (double)a / b);
    printf("%.2f\n", (double)(a / b));
    return 0;
}
```

Trace & Predict 2: Width and alignment

What does this print? Show spaces with a dot (.) so alignment is clear.

```
#include <stdio.h>
int main(void) {
    printf("[%5d] [%-5d] [%05d]\n", 42, 42, 42);
    return 0;
}
```

Find the Bug 1: Wrong format specifier and missing &

List every bug, explain why each is wrong, and write the corrected program.

```
#include <stdio.h>
int main(void) {
    float price;
    int qty;
    printf("Enter price and quantity: ");
    scanf("%d %d", &price, qty);
    printf("Total = %d\n", price * qty);
    return 0;
}
```

Write Code on Paper 1: Reverse a 4-digit number

Write a complete program that reads a 4-digit integer (1000–9999) and prints the number with its digits reversed (e.g. input 1234 $\Rightarrow$ output 4321). Use only / and %.

Write Code on Paper 2: Lowercase to uppercase, one character

Write a complete program that reads a single character that you may assume is a lowercase letter and prints the corresponding uppercase letter. Use arithmetic on ASCII codes (not the

`ctype.h` library).